

PRESCRIPTO

Comprehensive Smart Healthcare Platform

End-to-End Implementation & Architecture Report (Phases 1-6)

1. Architecture & Technology Stack

The Prescripto platform utilizes a decoupled architecture to separate a responsive frontend interface from a high-performance, AI-native backend handling heavy machine learning tasks, concurrent connections, and geospatial logic.

Domain	Technology	Implementation Purpose
Frontend	Next.js (React) + TailwindCSS	Provides server-side rendering and dynamic UIs for complex dashboards and mapping components.
Backend API	FastAPI (Python)	Native ML integration, delivering highly concurrent and asynchronous routing endpoints.
Database	Neon DB (Serverless PostgreSQL)	Handles relational data alongside native pgvector support for RAG embeddings.
AI & NLP	LlamaIndex + OpenAI	Orchestrates document ingestion, embedding generation, and contextual query processing for triage.
Mapping & Logic	Ola Maps API / Haversine	Geospatial math for calculating Distance Matrix ETAs and real-time ambulance tracking.
Storage		

Domain	Technology	Implementation Purpose
	AWS S3 / Supabase	Secure, scalable cloud blob storage for patient medical PDFs.
Notifications	Twilio API	Automated SMS delivery utilizing FastAPI background tasks.

2. Completed Development Phases

Phase 1: Foundation & Data Engineering

- **Synthetic Dataset Generation:** Created a highly realistic dataset (`synthetic_hospitals.xlsx`) modeling 12 prominent Delhi hospitals, complete with strict geographical coordinates, varying budget tiers (Low, Medium, Premium), and randomized available bed capacities.
- **Database Initialization:** Spun up Neon Serverless PostgreSQL, structurally defining SQLAlchemy schemas for Users, Hospitals, Appointments, and Medical Reports.
- **Vector Enablement:** Activated the `pgvector` extension within the database to prepare the architecture for AI-driven document embeddings.
- **Ingestion Pipeline:** Developed automated Python scripts (using `pandas` and `openpyxl`) to parse the Excel data and seed the PostgreSQL hospital tables.

Phase 2: User Access & Record Management

- **Secure Authentication:** Implemented a robust Pydantic-validated registration and login workflow. Passwords are securely hashed (`bcrypt`), and FastAPI generates secure JSON Web Tokens (JWT) for subsequent API authorization.
- **Cloud Report Storage:** Integrated AWS S3 using the `boto3` SDK. The backend securely receives PDF medical reports, streams them into the S3 bucket, and stores the permanent retrieval URLs directly in the patient's database profile.
- **Provider Portal Readiness:** Established data schemas allowing consulting doctors to retrieve and view these unified patient records securely.

Phase 3: The "One-Button" Emergency Engine

- **Condition & Budget Filtering:** Built a sophisticated predictive algorithm that takes incoming patient distress signals and filters the PostgreSQL hospital database strictly by required medical specialty, available budget, and real-time bed capacity.
- **Geospatial Routing Engine:** Integrated distance mathematics (Haversine formula/Ola Maps API) to dynamically calculate the absolute fastest route from the patient's coordinates to the filtered hospitals.
- **Logistics Dispatching:** Developed endpoints that output universal deep links, empowering the frontend to automatically open Uber or Ola mobile applications pre-filled with the calculated optimal hospital's drop-off coordinates.

Phase 4: RAG Chatbot & Automated Triage

- **Knowledge Base Assembly:** Gathered extensive medical triage guidelines into standard text files and constructed an automated Python ingestion script.
- **Embedding Pipeline:** Utilized LlamaIndex and OpenAI to translate standard medical documentation into vector embeddings, saving them permanently within Neon DB's pgvector tables.
- **Context-Aware Triage:** Constructed a FastAPI endpoint that retrieves relevant medical context based on patient symptoms. The LLM acts as a triage nurse, cross-referencing guidelines to evaluate urgency.
- **Structured JSON Output:** Enforced strict JSON schema outputs from the LLM to return exact recommended medical tests (e.g., Blood Count, ECG) to power frontend UI buttons seamlessly.

Phase 5: Standard Appointments & Notifications

- **Elective Routing Recommendations:** Adapted the Phase 3 algorithm for non-emergencies, actively prioritizing budget alignment and clinical specialty over sheer distance.
- **Transaction Locking (Double-Booking Prevention):** Leveraged strict PostgreSQL Row-Level Locking (`SELECT . . . FOR UPDATE`) on the booking endpoint. This guarantees database integrity by rejecting concurrent requests attempting to book the exact same doctor slot simultaneously.

- **Asynchronous SMS Notifications:** Integrated the Twilio SDK. Upon successful booking, FastAPI immediately responds to the user while silently spinning off an asynchronous Background Task to send SMS confirmations without blocking the main API thread.

Phase 6: Real-Time Fleet & Bed Tracking

- **Dynamic Bed Occupancy Simulation:** Written and deployed a continuous `asyncio` background script running within the FastAPI application lifecycle. It autonomously fluctuates hospital bed counts randomly to mimic a live, breathing hospital admissions environment.
- **Live Ambulance Telemetry (WebSockets):** Built high-performance WebSocket connections (`/ws/ambulance-tracker`) that push simulated, steadily updating GPS coordinates back to the frontend every two seconds.
- **Live Map Readiness:** Configured the continuous telemetry stream to format data identically to what the frontend Ola Maps component requires to render smooth, real-time ambulance movement on the screen.